

will have different back-ends for platform-specific machine code.

The optimiser is part of the compiler back-end. Recall that the intermediate code is taken as input by the back-end. Optimisations performed in the back-end can be either machine-independent or machine-dependent. Typically, the optimiser consists of two parts: a high-level optimiser, which performs machine-independent optimisations, and a low-level optimiser, which performs the machine-dependent optimisations.

Next, let us look at a few of the common optimisations performed by most compilers.

Where did all my code vanish?

Consider the following code snippet:

```
int foo(int x) {
    int i = 9;
    x = 0;
    if (x) {
        i = 13;
    }
    return i;
}
```

Now, when this function is optimised by the compiler, here is the high-level language version of the machine code that would be generated.

```
int foo(int x) {
    return 9;
}
```

The transformation of 'foo' from high-level to low-level, where it simply returns the value 9, involves two important optimisations: constant propagation and dead code elimination. In constant propagation, the compiler determines which of the variables involved in a computation are constant, and then uses the constant values to compute the result. If a variable has a constant value assigned to it, then the constant value is propagated to all uses of that variable. In the above code snippet, *x* was assigned the constant value 0. The compiler propagates this value to all uses of *x*. Thus, the 'if' condition becomes 'if (0)', which always evaluates to false. Now, by dead code elimination, the compiler eliminates the code controlled by the 'if' statement, since the condition always evaluates to false (the block will never be entered). Therefore, the only definition of 'i' that reaches the statement 'return i' is the assignment 'i = 9'. That's why the compiler again performs constant propagation for the variable 'i', reducing the function to the single statement 'return 9'.

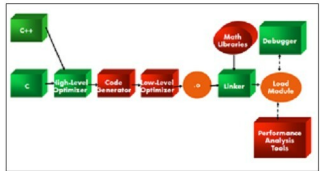


Figure 1: Typical compiler organisation

It is important to know the difference between dead code and useless code. Dead code is code that can never be executed. In the above code example, the statement 'i = 13' is dead code, since it cannot ever be executed. On the other hand, useless code consists of executable statements, but whose results are not used further in the computation, and hence can be eliminated if it does not have any other side effects. For example, consider the following code snippet:

```
int foo(int x)
{
    int i = 9;
    int j = 13;
    int k;
    k = i + j;
    k = k - 1;
    return i;
}
```

Here, the second to fifth statements are unused code, since their results are not used further in the function. Hence, the compiler can eliminate these useless code statements.

Another important optimisation that compilers perform is redundant computation elimination. Redundant computation is when the result value of a computation is already available at the point of computation. In other words, at a given program point *P*, an expression *X op Y* is redundant if there is another evaluation of *X op Y* that definitely reaches this point *P* along all paths from the program entry, and there is no re-definition of the operands *X* and *Y* from the earlier evaluation of the expression along any of the paths to *P*.

There are two types of redundant computation elimination. One is fully redundant computation elimination, more popularly known as 'common sub-expression elimination'. Another is partial redundant elimination (PRE). Let's look at examples of each of these optimisations.